Where Should the Network Look Next? Multi-Objective Measurement Control for Programmable Network Monitoring

Paper #XXX [Frontiers Track]

March 13, 2026

Abstract:

Modern networks can measure their own traffic in many ways: polling counters, sampling packets, running compact data structures called sketches, and embedding telemetry headers directly into packets. But each monitoring system built so far optimizes only one thing—communication cost, switch memory, detection accuracy, or estimation quality—while ignoring the others. When a real network needs all of these at once, the systems conflict.

We analyze eighteen published monitoring systems spanning 2008–2024, trace how the field evolved from fixed switches to fully programmable pipelines, and identify the architectural contradictions that arise when single-objective designs meet multi-objective reality. From this analysis, we propose MCP (Measurement Control Plane): a subsystem that lives in the control plane and decides, in real time, what to measure, where to measure it, and how to process the result—under hard limits on switch memory, controller CPU, and control-channel bandwidth. MCP uses an online learning algorithm (a constrained contextual bandit) to allocate measurement resources across competing tasks. We describe the architecture, the algorithm, and the evaluation methodology needed to test whether better monitoring actually makes the network work better.

1 Introduction

A network that cannot see its own traffic cannot manage it. Operators need to know which links are congested, which flows are misbehaving, and whether an attack is underway. Monitoring provides this visibility. Without it, the control plane is flying blind—it cannot reroute traffic around a failure it does not know about, and it cannot block an attack it cannot detect.

In traditional networks, monitoring was rigid. Routers had fixed counters and supported fixed sampling protocols like NetFlow [6]. Operators could read the counters or enable sampling, but they could not change *what* the router measured or *how* it measured. The hardware vendor decided what was observable, and operators worked within those limits.

Software-defined networking (SDN) changed this. OpenFlow [12] separated the control plane from the data plane, giving a remote controller the ability to install flow rules and read per-rule counters. For the first time, the controller—not the switch vendor—could decide which flows to track and when to ask for statistics. This made monitoring programmable, but only in a limited way: the switch could count packets that matched a rule, but it could not run more complex operations like hashing or sketching.

P4 [4] went further. Instead of a fixed set of protocol headers, P4 lets programmers define how the switch parses, matches, and acts on packets. This means monitoring logic itself—not just what flows to count, but *how* to count them—can be written as a program and loaded onto the switch hardware. The switch becomes a programmable measurement platform.

With these two shifts (programmable control plane, then programmable data plane), the monitoring question changed. It used to be "can we see the traffic?" Now it is "*what should we look at*, given that every measurement we run costs something?" Polling a counter costs control-channel bandwidth because the controller must send a request and wait for a response. Running a sketch—a compact probabilistic data structure that estimates traffic statistics—costs register memory and pipeline stages inside the switch hardware. Sampling packets costs mirror bandwidth and CPU at the collector. Every resource spent on monitoring is a resource not available for forwarding packets.

Over the past decade, researchers have attacked this cost-vs-visibility problem from many angles. Some made polling cheaper [20, 5, 21]. Some chose better locations to monitor [17, 22]. Some built smarter data structures for the switch pipeline [11, 13, 14, 16]. Some split the work between the switch and an external stream processor [10].

Each of these contributions works within its own scope. But none of them answers the combined question: if QoS monitoring, traffic estimation, and attack detection all need measurements *from the same switch at the same time*, which

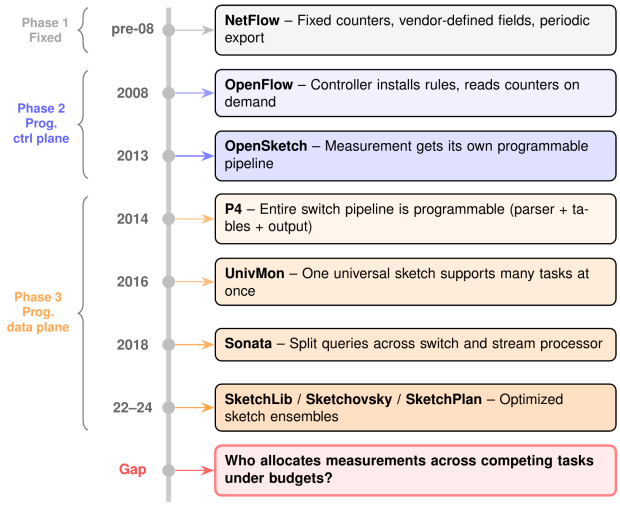

Figure 1: Three phases of network monitoring. Phase 1 offered fixed measurement. Phase 2 made the control plane programmable. Phase 3 made the data plane programmable. The gap at the bottom—deciding which measurements to run under competing demands—is what this paper addresses.

measurements should the switch run? Who decides? And how do they decide without exceeding the switch's hardware limits?

That is the question this paper addresses. We propose MCP, a Measurement Control Plane that **lives in the control plane** (not on any switch) and continuously allocates monitoring resources across tasks, switches, and processing planes. Figure 3 shows exactly where MCP sits in the network stack.

The rest of this paper explains why the problem is hard (Section 2, Section 3, Section 4), how we propose to solve it (Section 5), and how to test whether the solution works (Section 6).

2 Background: How Network Monitoring Evolved

This section traces the key ideas that brought us to the current state of network monitoring. Figure 1 summarizes the three phases. Readers already familiar with SDN and P4 can skip to Section 3.

2.1 Phase 1: Fixed monitoring (pre-2008)

Before SDN, network devices came with built-in counters and sampling mechanisms that the operator could turn on or off but could not fundamentally change. NetFlow [6], introduced by Cisco in the 1990s, is the best-known example. It recorded per-flow statistics—source and destination addresses, port numbers, protocol type, packet counts, byte counts—and exported them periodically to a collector. The operator could configure how often to export and what sampling rate to use, but the set of fields the router could observe was fixed in silicon. If an operator wanted to track a metric that NetFlow did not support (say, per-packet latency or entropy of destination addresses), there was no way to do it without replacing the hardware.

This rigidity was a direct consequence of the hardware design philosophy of the era: every protocol and every measurement function was burned into the chip at manufacturing time.

2.2 Phase 2: Programmable control plane (2008–2013)

OpenFlow [12] introduced a different model. Instead of each switch making its own forwarding decisions based on built-in logic, a central controller installed rules into the switch's flow table over a secure channel. Each rule said: "if a packet matches this pattern, do this action (forward to port X, drop, send to controller)." Crucially, each rule also had counters that recorded how many packets and bytes had matched it.

This gave monitoring two new capabilities that NetFlow lacked. First, the controller could install rules *specifically* for measurement purposes—rules whose only job was to match a set of flows and count them. Second, the controller could change those rules at any time, adding new ones when suspicious traffic appeared and removing old ones when they were no longer needed. Monitoring became dynamic rather than static.

But OpenFlow switches were still limited in what they could do with each packet. They could match on header fields and count matches, but they could not run more complex operations. A switch could tell you how many packets matched rule 42, but it could not compute a hash of the source address, update a probabilistic counter, or embed a timestamp into the packet header. For anything beyond simple counting, the data had to be sent to the controller or a separate collector for processing.

OpenSketch [23] was one of the first systems to recognize that measurement needed its own programmable pipeline, separate from forwarding. It proposed a three-stage architecture—hash, filter, count—implemented on NetFPGA hardware, with a control-plane library that automatically configured the pipeline for different measurement tasks (heavy hitter detection, change detection, flow size estimation). The key insight was that measurement is itself a resource allocation problem: different tasks need different amounts of memory and computation, and the system must decide how to divide limited hardware resources among them. This idea is a direct ancestor of the work we propose.

2.3 Phase 3: Programmable data plane (2014–present)

P4 [4] generalized OpenSketch's insight from measurement to the entire switch pipeline. Where OpenSketch provided a fixed three-stage pipeline for measurement, P4 made everything programmable: how the switch parses incoming packets (which headers to extract), how it matches them (which fields to compare against which tables), and what actions to take (forward, drop, modify headers, update state). A programmer writes a P4 program that defines all of this, compiles it, and loads it onto the switch hardware.

For monitoring, this was transformative. A P4 program can define custom data structures called *sketches*—compact arrays that use hash functions to map packet fields to counters, allowing approximate but memory-efficient tracking of traffic statistics. It can implement *in-band network telemetry* (INT), where each switch appends a small metadata header to selected packets as they pass through, recording queue depth, processing latency, and other per-hop information. And it can do all of this at line rate—billions of packets per second—because the logic runs in the switch's forwarding pipeline, not on a general-purpose CPU.

This phase produced a rapid series of advances in what the switch can do for measurement.

UnivMon [11] showed that a single "universal sketch" can support many different monitoring tasks (heavy hitters, entropy estimation, flow size distribution, change detection) with accuracy comparable to custom per-task algorithms. The key idea is that instead of running a separate sketch for each task, you run one general-purpose sketch and compute different estimates from it at query time. UnivMon demonstrated a P4 implementation and a control plane that coordinates sketches across multiple switches.

Sonata [10] introduced query-driven telemetry. Operators write monitoring queries in a high-level language ("find all hosts with more than 100 new TCP connections in the last minute"), and the system automatically decides how much of each query to run on the switch (at line rate) and how much to send to a stream processor (which is slower but more flexible). This partitioning reduced stream-processor load by up to seven orders of magnitude.

SketchLib [13] analyzed why sketches are often infeasible on real switch hardware even though they work fine in simulation. The problem is that hardware switches have specific constraints—limited stateful ALUs (arithmetic logic units that can update counters), limited hash units, and a fixed number of pipeline stages—and naive sketch implementations waste these resources. SketchLib identified five categories of bottlenecks and provided optimization techniques that cut resource usage by up to 96%.

Sketchovsky [14] extended this to *ensembles* of sketches. A network operator who needs to track heavy hitters, estimate flow entropy, and detect superspreaders simultaneously would normally need three separate sketches, each consuming its own hash units, counters, and pipeline stages. Sketchovsky showed that these sketches can share resources—reuse hash computations, merge counter arrays, share key-field extraction—making ensembles of up to 18 sketch instances feasible on a single Tofino switch where only a few would fit without sharing.

HeteroSketch [2] tackled the network-wide version of this problem: given a network of switches with *different* capabilities (some programmable, some fixed-function, some with more memory than others), how do you decide which device runs which measurement task? Its profiling-based optimizer matched tasks to device capabilities and reduced resource overhead by 20–60%.

SketchPlan [16] raised the abstraction further. Instead of asking operators to choose specific sketches and configure their parameters, SketchPlan lets them specify measurement *intents*: "I want heavy-hitter detection with 95% recall and 3% relative error." The system automatically selects the right sketches, configures them, and takes an ensemble view that jointly optimizes across multiple intents. This improved accuracy–resource tradeoffs by up to 60× compared to optimizing each intent in isolation.

OctoSketch [24] addressed a different bottleneck: software switches that process packets across multiple CPU cores. Merging sketch results from different cores introduces errors and computation overhead. OctoSketch devised a continuous, change-based aggregation mechanism that reduced errors by 15.6× and improved throughput by up to 4.5×.

2.4 What is still missing

Each of these systems solves one piece of the puzzle. The sketch papers tell us *how* to run measurements efficiently on hardware. The polling papers tell us *when* to collect statistics. The placement papers tell us *where* in the network to put monitors. But nobody answers the combined question: given a fixed budget of switch memory, controller CPU, and bandwidth, and given multiple applications that all want measurements at the same time, *which measurements should actually run?*

Figure 2 makes this gap concrete. It maps eleven systems against five decision dimensions. No prior system covers more than two.

3 What the Literature Got Right—and Where It Disagrees

We analyzed eighteen systems spanning 2008–2024 and organized them not by publication date but by what they optimize and what they assume.

3.1 What the field agrees on

Four claims are supported across the entire corpus.

System	When (poll freq.)	Where (switch sel.)	How (primitive)	Process (location)	Joint (budgets)
OpenNetMon	✓	–	–	–	–
PayLess	✓	–	–	–	–
FlowCover	–	✓	–	–	–
Xu et al.	✓	–	✓	–	–
Yoon et al.	–	✓	–	–	–
Tian et al.	–	–	✓	–	–
FlowSpy	–	✓	✓	–	–
Sonata	–	–	✓	✓	–
Sketchovsky	–	–	✓	–	–
SketchPlan	–	–	✓	–	–
HeteroSketch	–	✓	✓	–	–
MCP (ours)	✓	✓	✓	✓	✓

Every prior system covers **at most 2 columns**. MCP is the first to address all five decisions jointly under explicit resource budgets.

Figure 2: What each system decides. *When* = polling frequency. *Where* = which switches. *How* = which primitive (counter, sketch, sample, INT). *Process* = where to compute (switch vs. controller vs. stream processor). *Joint* = multi-objective allocation under budgets. Every prior system covers at most two columns. MCP covers all five.

First, **brute-force monitoring does not scale.** Polling every counter at high frequency floods the control channel with request-response messages. Installing a measurement rule for every individual flow exhausts the TCAM (Ternary Content-Addressable Memory), which is the scarce, power-hungry memory that stores matching rules inside the switch. Every paper in this corpus either demonstrates or assumes this constraint.

Second, **being selective helps.** Whether by adapting polling intervals so that stable flows are checked less often [20, 5], choosing which switches to poll [17], picking only measurements that add new information to a traffic-matrix estimate [19], or selecting sampling points by betweenness centrality [22], the field agrees that targeted monitoring outperforms exhaustive monitoring.

Third, **the value of a measurement depends on the task.** A per-flow byte counter is useful for QoS monitoring but irrelevant for DDoS detection, which cares more about the number of distinct source addresses per destination. A traffic-matrix estimator needs measurements that are linearly independent from measurements already taken, while an IDS needs measurements from switches that see the widest variety of traffic. Different tasks need fundamentally different measurement strategies.

Fourth, **programmability changes everything.** Each generation of programmability—OpenFlow (programmable control plane), OpenSketch (programmable measurement pipeline), P4 (programmable data plane)—opened new ways to measure. The latest generation (P4 with sketch libraries) makes it physically possible to run many measurement tasks on one switch, but this creates a new allocation problem: which tasks get which resources?

3.2 Three research directions

The eighteen systems cluster into three directions.

Direction 1: Make polling cheaper. OpenNetMon [20] and PayLess [5] adapt polling frequency: poll more often when traffic changes rapidly, less often when it is stable. Xu *et al.* [21] reduce the number of requests by using wildcard rules that aggregate multiple flows into a single counter, cutting both control-channel bandwidth and switch processing delay.

Direction 2: Choose better measurement targets. FlowCover [17] picks which switches to poll using a weighted set-cover optimization. Tian *et al.* [19] pick which flows to measure based on whether each new measurement increases the rank of the traffic-matrix estimation system—if it does not, the measurement is redundant and wastes a TCAM entry. FlowSpy [9] assigns monitoring tasks to P4 switches with explicit load balancing. HeteroSketch [2] places sketches across heterogeneous devices by profiling each device's capabilities first. Tahaei *et al.* [18] redesign measurement for the practical reality of distributed controllers, where synchronization overhead becomes a first-class cost.

Direction 3: Make the switch pipeline do more. This direction, described in Section 2, includes OpenSketch [23], UnivMon [11], Sonata [10], SketchLib [13], Sketchovsky [14], SketchPlan [16], and OctoSketch [24]. For security, Yoon *et al.* [22] choose sampling points by centrality, and El Sayed *et al.* [7] build SDN-native DDoS classifiers using features extracted from flow statistics.

3.3 Assumptions nobody tests

We found six assumptions that most papers rely on without experimental validation.

A1: Switch counters are accurate enough to optimize around. All polling-based systems use switch-reported counters as ground truth for their optimization decisions. But switch counters can be stale (updated at different rates than the polling interval), delayed (buffered before export), or affected by internal scheduling artifacts. If the counters are noisy at timescales that matter for the optimization, the cost–accuracy tradeoffs these papers report may not hold in practice.

A2: The cost–accuracy tradeoff is smooth. The literature implicitly assumes that if you reduce measurement (poll less often, use fewer rules, run smaller sketches), accuracy degrades gradually and predictably. But if the tradeoff has discontinuities—sharp cliffs where a small reduction in measurement causes a large drop in visibility—then the smooth

tradeoff curves in these papers overstate the practical operating range.

A3: Partial observation catches what matters. Selective monitoring works only if the "right" subset of flows to observe does not change faster than the system can adapt its selection. If an attack appears on a flow that the system decided not to monitor, partial observation misses exactly what it was designed to catch.

A4: The controller has spare capacity for monitoring. Even systems that reduce polling overhead assume the controller can absorb the remaining monitoring workload without affecting its primary job of managing forwarding rules. If monitoring and control compete for the same CPU and memory on the controller machine, monitoring overhead becomes a control-plane stability risk.

A5: Lab results transfer to production. Most evaluations use Mininet (a software emulator) with small topologies and synthetic traffic. SketchLib [13] showed that sketch implementation details—the specific way hash functions are assigned to hardware units, how counter widths interact with stateful ALU constraints—materially change accuracy on real Tofino hardware compared to software simulation. This is a direct warning that lab results may not predict deployment behavior.

A6: Better measurement means better network outcomes. This is the most consequential assumption. Almost every paper in this corpus stops at measurement quality: lower polling cost, higher estimation accuracy, better detection rates. None of them closes the loop to show that the downstream application—the traffic engineering controller that reroutes flows, the IDS that blocks attackers, the QoS manager that adjusts priorities—actually makes better decisions as a result of the improved monitoring. It is entirely possible that a monitoring system with excellent internal metrics produces telemetry that the downstream application cannot use effectively.

4 Where the Papers Contradict Each Other

Not every difference between systems is a disagreement. Many papers simply study different problems. But four differences are genuine contradictions—positions that cannot both be true in the same deployment.

Contradiction 1: Where should monitoring logic live?. The OpenFlow-era papers [20, 5, 17] place monitoring intelligence in the controller. The controller decides what to measure, sends requests to the switch, collects responses, and processes the results. The switch is passive—it counts when asked. FlowSpy [9] and the sketch papers [14] take the opposite position: monitoring tasks should run directly in the P4 switch pipeline, with the controller involved only for task assignment, not for data collection. El Sayed *et al.* [7] take a third position: detection should stay at the controller because putting decision-making logic in the switch violates SDN's principle of separating forwarding from control.

These three positions cannot all be correct for the same network. The disagreement exists because each group assumed different hardware. The OpenFlow papers assumed switches that can only count packets. The P4 papers assumed switches that can run arbitrary programs. El Sayed *et al.* assumed OpenFlow-era switches where in-switch intelligence is not feasible. The contradiction is not about philosophy—it is about which hardware generation the authors had in mind.

Contradiction 2: Where in the network should monitors sit?. OpenNetMon monitors at edge switches because they sit at flow endpoints, making per-flow measurement straightforward. Yoon *et al.* [22] monitor at high-centrality switches because those switches see the most diverse traffic, which is better for feeding an IDS. HeteroSketch [2] says the answer depends on each device's hardware capabilities, not just its topological position.

None of these papers considers what happens when QoS monitoring (which favors edge switches) and security monitoring (which favors high-centrality switches) must both run on the same network with the same set of switches.

Contradiction 3: Does measuring more always help?. PayLess assumes that finer-grained, higher-frequency collection improves monitoring quality. This is true for operational monitoring, where more data generally means more accurate statistics. Tian *et al.* [19] explicitly reject this for traffic-matrix estimation: a measurement that does not increase the rank of the linear system that maps observable flows to the full traffic matrix contributes zero new information. Installing it wastes a TCAM entry that could have been used for a measurement that actually adds something.

This is not a minor disagreement. It reflects fundamentally different mathematical structures: operational monitoring has diminishing returns (more data helps, but with decreasing marginal benefit), while inverse estimation has a rank condition (some measurements are completely redundant regardless of how many you already have).

Contradiction 4: Is a single controller enough?. The early papers assume a single logically centralized controller is both sufficient and beneficial—it provides a global view that makes monitoring easier to orchestrate. Tahaei *et al.* [18] argue that in practice, a single physical controller creates scalability, reliability, and performance bottlenecks. El Sayed *et al.* [7] go further: even multiple controllers do not help for DDoS defense, because the controllers themselves can be targeted by the attack.

What these contradictions mean for MCP. Each system picks one objective, designs an architecture that serves it well, and treats the other objectives as someone else's problem. A deployed network has QoS monitoring, traffic estimation, attack detection, and more, all running at the same time on the same hardware. The contradictions arise because

nobody has built the layer that manages these competing demands. That is the gap MCP addresses.

5 Proposed Design: A Measurement Control Plane

MCP is a subsystem that **lives in the control plane**, alongside the forwarding controller. It does not run on any switch. It reads switch state over standard interfaces, makes allocation decisions on general-purpose hardware, and writes measurement configurations back to switches. Figure 3 shows exactly where it sits.

5.1 Why the switch itself is the bottleneck

To understand why measurement allocation is hard, consider what happens inside a P4 switch. Figure 4 shows the key resources.

A P4 switch processes packets through a series of *pipeline stages*. Each stage is a self-contained hardware block that can match on packet headers and perform actions. The number of stages is fixed at chip fabrication—a typical Tofino switch has 12 ingress stages and 12 egress stages. You cannot add more.

Within each stage, three types of resources are available. **TCAM** (Ternary Content-Addressable Memory) stores matching rules. A forwarding rule ("if destination IP matches 10.0.0.0/24, send to port 3") uses TCAM entries. A measurement watchlist rule ("if source IP is in the suspicious set, increment a counter") also uses TCAM entries. Both compete for the same finite memory. **Registers** store persistent state. A forwarding application might use registers to track connection state. A sketch data structure uses register arrays to store approximate counters. Both compete for the same on-chip SRAM. **ALUs** (Arithmetic Logic Units) perform computation. Forwarding logic might use an ALU to compute an ECMP hash for load balancing. A sketch uses ALUs to compute hash functions for updating counters. Both compete for the same hardware units.

When MCP loads a sketch array into Stage 3, those registers are no longer available for forwarding state. When it installs a watchlist rule in Stage N, those TCAM entries are no longer available for routing rules. This is why measurement allocation is a resource problem, not just a configuration choice: every measurement task takes a piece of the fixed pipeline, and the pipeline cannot grow.

5.2 Architecture overview

MCP has three layers, shown in Figure 3.

Layer 1: Data plane (the switches). Each P4 switch runs measurement primitives: sketch arrays, per-flow counters, sampling/mirroring with meter-based rate caps, and optional INT headers. These are compiled into the switch pipeline using the Portable Switch Architecture (PSA) [15]. The switch reports its current resource usage—how many TCAM entries are in use, how much register memory is allocated, which pipeline stages are committed—to MCP via gNMI (gRPC Network Management Interface), a standard protocol for streaming device telemetry.

Layer 2: Measurement Control Plane (the brain). This is the core of the system. It runs on general-purpose hardware in the control plane—the same server rack or cluster where the forwarding controller runs. It has four components that execute in sequence every epoch (a configurable interval; 1 second is a reasonable starting point):

A *context monitor* reads the current network state via gNMI streams: the traffic mix (how many large flows vs. small flows, the rate of new flow arrivals, which prefixes are most active), security signals (anomaly scores from the IDS, alert flags from upstream sensors), topology information (which switches are connected to which, betweenness centrality values), and resource utilization (current TCAM occupancy, register usage, controller CPU load, control-channel bandwidth consumption).

A *candidate generator* produces a bounded list of possible measurement actions. Each action is a four-part tuple: (1) what to observe (a prefix, a 5-tuple class, a set of suspicious flows), (2) which primitive to use (counter, sketch, sampled packets, INT metadata), (3) at what rate (polling interval, sampling probability, trigger threshold), and (4) where to process the result (on-switch, at the controller, or in a stream processor). The generator draws on techniques from the whole literature: set-cover placement from FlowCover [17], wildcard aggregation from Xu *et al.* [21], centrality-based sampling from Yoon *et al.* [22], sketch ensemble configuration from Sketchovsky [14], and query partitioning from Sonata [10]. MCP's contribution is not another measurement technique. It is the allocator that picks among existing techniques.

A *multi-objective selector* chooses which actions to deploy. This is where the optimization happens. Every epoch, it maximizes:

$$U(S_t) = w_{\text{acc}} \cdot \text{Acc}_t - w_{\text{lat}} \cdot \text{Lat}_t - w_{\text{chg}} \cdot \text{Churn}(S_t, S_{t-1}) \quad (1)$$

subject to hard budget constraints:

$$\text{BW}_t \leq \overline{B},\ \text{TCAM}_t \leq \overline{T},\ \text{REG}_t \leq \overline{R},\ \text{CPU}_t \leq \overline{C} \quad (2)$$

In plain language: maximize the accuracy of all monitoring tasks and minimize detection latency, while penalizing frequent reconfiguration (which disrupts measurement continuity), and never exceed the operator-specified limits on control-channel bandwidth, TCAM entries, register memory, or CPU usage. The weights w_{acc}, w_{lat}, w_{chg} let operators express priorities: a security-first deployment would increase w_{lat}; a bandwidth-constrained deployment would keep tighter $\overline{B}$.

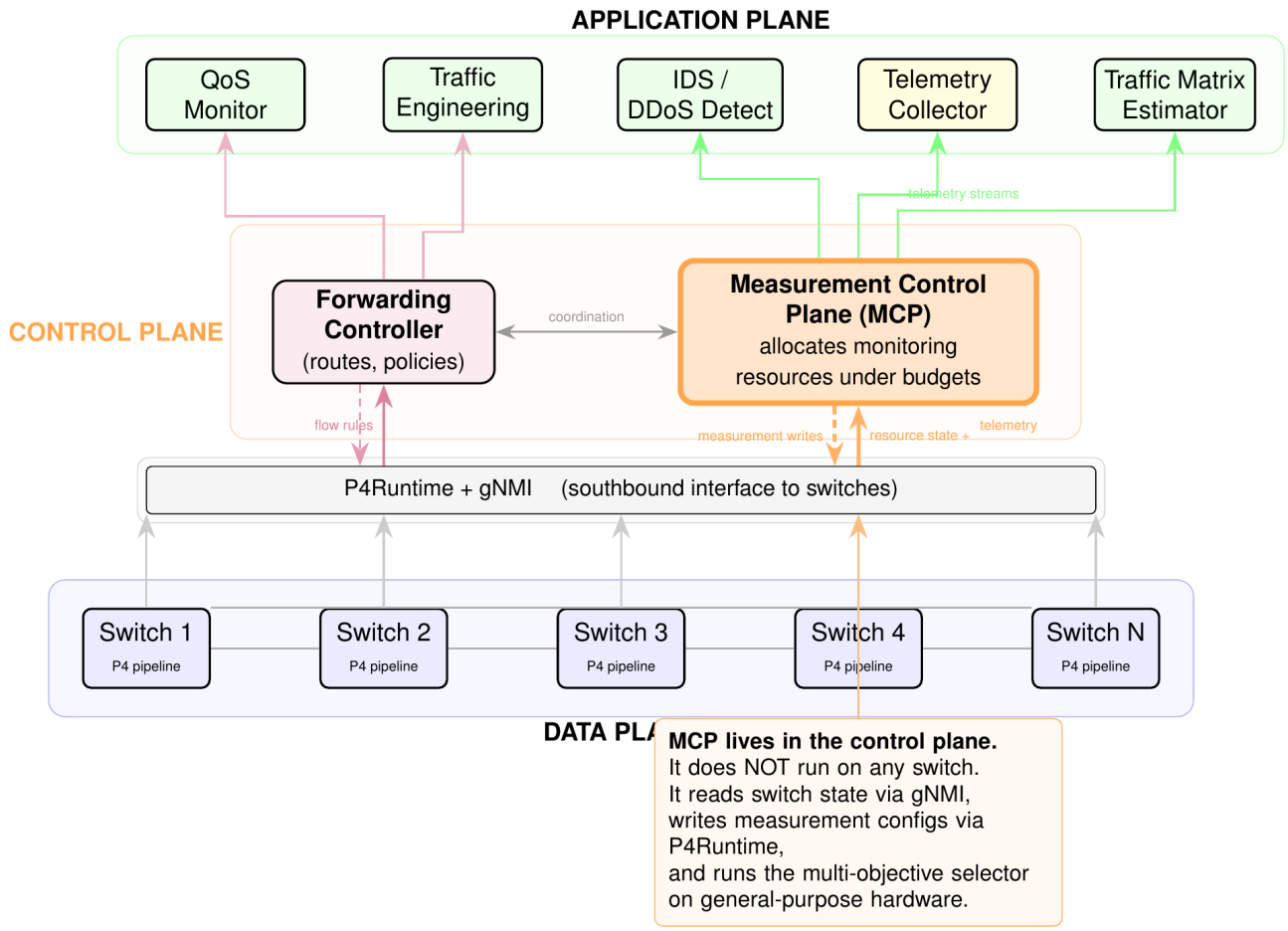

Figure 3: Where MCP lives in the network. MCP sits in the **control plane**, alongside the forwarding controller. It is not part of any switch. It reads switch resource state via gNMI (a standard telemetry streaming protocol), makes allocation decisions on general-purpose hardware, and writes measurement configurations to switches via P4Runtime (a standard switch programming API). The forwarding controller handles routing and policies; MCP handles measurement. They coordinate but are separate services.

A *deployer* translates the selected measurement plan into P4Runtime table writes, register parameter updates, meter configurations, and gNMI subscription changes. It uses atomic plan versioning: either the entire plan deploys successfully, or it rolls back to the previous plan. This prevents partial configurations where some switches have the new plan and others still run the old one.

Layer 3: Analytics and actuation. A collector ingests telemetry reports, packet digests, and sampled packets from the switches. A stream processor merges sketch results, computes QoS features, estimates traffic matrices, and runs IDS inference. An actuator applies mitigation actions—rate limiting, blackholing, rerouting—when the analytics layer detects a problem. Outcomes (did detection latency improve? did the false positive rate change? was the attack contained?) feed back to the selector as reward signals.

5.3 How the selector adapts: shadow prices

The selector uses a type of online learning algorithm called a *constrained contextual bandit*. To understand why, consider the three properties of MCP's allocation problem:

1. **It repeats.** Every epoch (every second), MCP must choose a new measurement plan. Traffic changes, attacks start and stop, resource availability shifts. A static plan computed once cannot keep up.

2. **It has multiple hard constraints.** The plan must not exceed budgets for TCAM, registers, bandwidth, and CPU. These are hard limits, not soft guidelines—

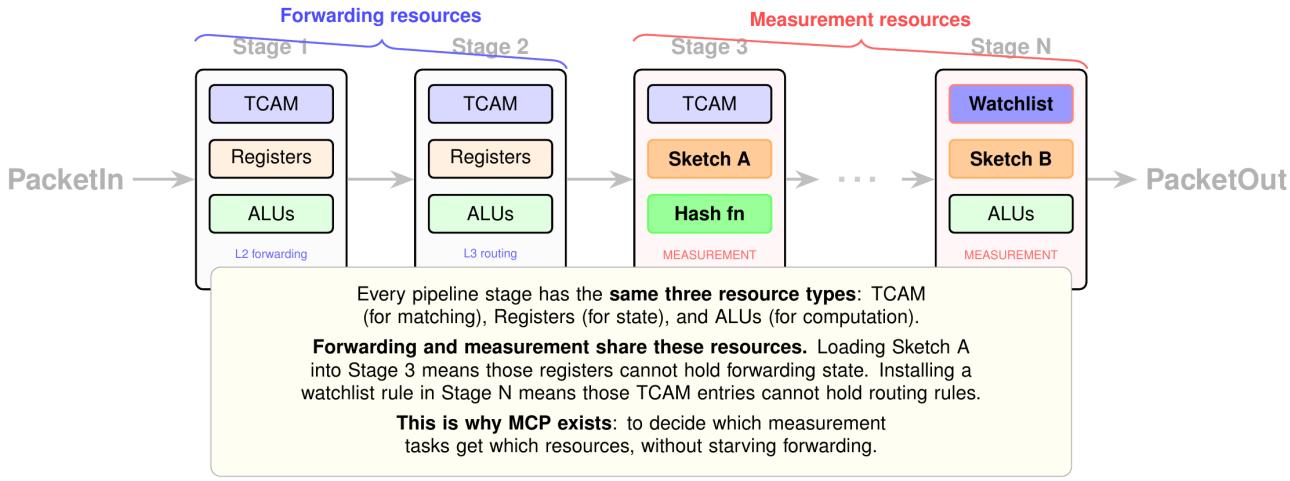

Figure 4: Inside a P4 switch pipeline. Packets flow left to right through a series of stages. Each stage has three resource types: **TCAM** (for matching packet headers against rules), **Registers** (for storing state like sketch counters), and **ALUs** (for computation like hash functions). Stages 1–2 are used for forwarding (blue annotation). Stages 3 and N show measurement tasks loaded by MCP (red annotation): a sketch array, a hash function, and a watchlist rule. These consume the same physical resources that forwarding uses. This is why measurement allocation is a resource competition, not just a design choice.

exceeding them means the switch cannot install the rules or the controller crashes.

3. **The rewards are unknown.** How much accuracy or latency improvement a given measurement action provides depends on the current traffic, which MCP does not know in advance. It must learn from experience.

The Bandits with Knapsacks framework [3] is designed for exactly this situation: online decision-making with unknown rewards and budget constraints. Recent work by Foster *et al.* [8] extends it to contextual settings with multiple packing constraints, which matches MCP's multi-budget structure.

The mechanism that makes this work in practice is *shadow prices*. Figure 5 illustrates the idea.

Each resource (TCAM, registers, bandwidth, CPU) has a shadow price λ_i that tracks how tight the budget is. When TCAM is 90% full, λ_{TCAM} rises, making any action that consumes TCAM entries—like installing a watchlist rule—look expensive in the selector's scoring function. The selector naturally shifts toward actions that use registers instead (like sketch arrays). When bandwidth has plenty of headroom, λ_{BW} drops, making polling-based actions (which consume bandwidth) look cheap.

This happens automatically every epoch. The update rule is simple: $\lambda_i \leftarrow \max(0, \lambda_i + \eta \cdot (\text{realized_cost}_i - \overline{B}_i))$. If the realized cost for resource i exceeds its budget, λ_i increases. If it is below budget, λ_i decreases. No operator intervention is needed.

Algorithm 1 gives the full pseudocode.

Algorithm 1 MCP-RT: Real-time measurement allocation

Require: Budgets $\overline{B}$; headroom ρ; value model $V(a|x;\theta)$; cost model $\hat{C}(a|x)$

Init shadow prices $\lambda \geq 0$, $S_{\text{prev}} \leftarrow \emptyset$
for each epoch t **do**
  $x_t \leftarrow$ observe context via gNMI
  $A_t \leftarrow$ generate candidates
  for each $a \in A_t$ **do**
    $\text{score}(a) \leftarrow V(a|x_t;\theta) - \lambda \cdot \hat{C}(a|x_t)$
  end for
  $R \leftarrow (1-\rho)\overline{B} - \text{current_usage}$
  $S_t \leftarrow \emptyset$; sort A_t by score/cost descending
  for each a in sorted order **do**
    if $\text{score}(a) > 0$ **and** $\text{fits}(a,R)$ **and** $\text{churn_ok}(a, S_{\text{prev}})$ **then**
      $S_t \leftarrow S_t \cup \{a\}$; $R \leftarrow R - \hat{C}(a)$
    end if
  end for
  Deploy S_t via P4Runtime + gNMI
  Observe outcomes y_t
  $\theta \leftarrow \text{bandit_update}(\theta, x_t, S_t, \text{reward}(y_t))$
  $\lambda_i \leftarrow \max(0,\ \lambda_i + \eta(\text{cost}_i - \overline{B}_i)) \quad \forall i$
  $S_{\text{prev}} \leftarrow S_t$
end for

5.4 Why not a simpler approach?

A natural question is: why not just solve an optimization problem every epoch, without the bandit machinery? The

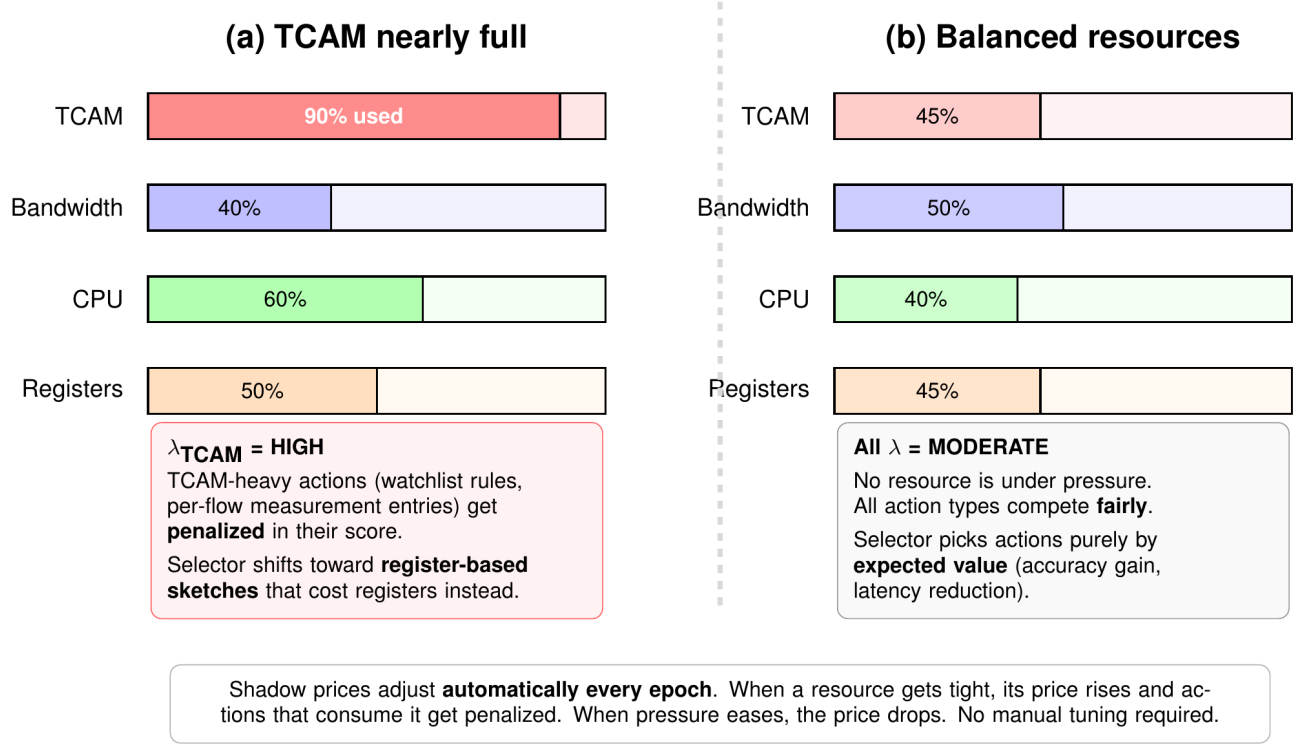

Figure 5: How shadow prices steer allocation. (a) When TCAM is 90% full, the shadow price λ_{TCAM} rises high, making TCAM-heavy actions (watchlist rules, per-flow entries) look expensive in the scoring function. The selector shifts toward register-based sketches, which consume registers instead of TCAM. (b) When all resources have headroom, shadow prices are moderate and the selector picks actions by expected value alone. Shadow prices adjust automatically every epoch. No manual rebalancing is needed.

answer is that the problem has two features that make static optimization insufficient. First, several subproblems are NP-hard: the set-cover placement from FlowCover and the wildcard scheduling from Xu *et al.* both require combinatorial search. Second, the reward function—how much accuracy or latency improvement each action actually provides under the current traffic—is unknown. The bandit learns it from experience.

A second question is: why not full reinforcement learning, which can handle long-term credit assignment? Because constrained RL (like CPO [1]) is heavier-weight and harder to safety-engineer for always-on production monitoring. If the RL agent explores poorly, it could exceed a budget and crash the switch. The constrained bandit with shadow prices is a practical middle ground: it learns, it respects budgets by construction, and it is lightweight enough to run every second.

6 How to Test Whether This Works

Testing MCP requires going beyond measurement quality. Figure 6 shows the full evaluation cycle.

Report all objectives simultaneously. Control-channel bandwidth, switch resource utilization (TCAM entries, pipeline stages, register memory), controller CPU, per-task measurement accuracy, and downstream application quality must all be reported together. A system that improves one metric while silently degrading another has not made progress—it has just moved the problem.

Run competing tasks. At minimum, run QoS estimation and DDoS detection simultaneously on the same switches. Single-task evaluations completely hide the allocation problem that MCP is designed to solve.

Close the loop. Connect MCP's telemetry output to at least one downstream application—a traffic engineering controller, a rate limiter, an alerting system—and measure whether that application's decisions improve compared to each baseline. If the downstream application does not improve, the monitoring system's internal metrics (accuracy,

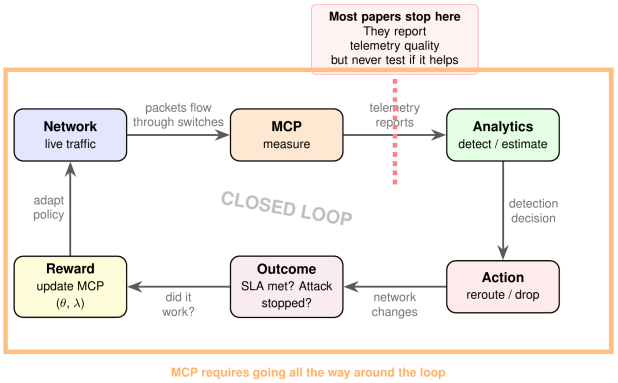

Figure 6: The closed-loop evaluation cycle that MCP requires. Most monitoring papers stop at the "Analytics" box: they report telemetry quality but never test whether the downstream application (rerouting, attack blocking) actually makes better decisions. MCP requires completing the entire loop: measuring, analyzing, acting, observing the outcome, and feeding the result back to update the measurement policy. Only by going all the way around can we test whether better monitoring makes the network work better.

latency, cost) are irrelevant.

Test on real hardware. At least one experiment must run on a P4 hardware target (Intel Tofino or equivalent) to confirm that the resource model is accurate and that measurement primitives operate at line rate under the proposed allocation.

Baselines. Each baseline represents one technique from the literature, running in isolation: (1) fixed periodic polling, (2) adaptive polling [20], (3) placement-only optimization [17], (4) wildcard aggregation [21], (5) centrality-based sampling [22], (6) query-driven partitioning (Sonata-style [10]), (7) sketch-based measurement (Sketchovsky [14]), and (8) fixed-policy INT. MCP should outperform each in multi-objective Pareto terms, not just on any single metric.

Testbed. Start with Mininet + Stratum BMv2 + ONOS for toolchain validation. Move to a Tofino-based hardware switch for resource-ceiling realism. Use CICIDS2017 and InSDN datasets for security evaluation, replayed into the SDN fabric with flow-level labels.

7 What Can Go Wrong

Table 1 lists the failure modes we anticipate and the mitigations designed into the architecture.

Table 1: Anticipated failure modes and mitigations.

Failure	Cause	Mitigation
Telemetry storms	INT triggers too broad; incident causes export flood	Per-switch meter caps; shadow prices spike for BW; shed low-value actions first
CPU saturation	Excessive polling or feature computation	Explicit CPU budget; offload to switch sketches or stream processor
TCAM exhaustion	Measurement rules crowd out forwarding	Hard cap; prioritize high-information candidates; fall back to register-based sketches
Reconfig churn	Frequent updates cause oscillation	Churn penalty in utility; max updates per epoch; plan versioning + rollback
Controller split	Delayed sync between distributed controllers	Raft-backed budget ledger; single-writer P4Runtime mastership per switch

8 Related Work

We organize related work by the layer of the monitoring stack each system addresses, and discuss each system's contribution and limitations in the context of the multi-objective allocation problem.

The measurement substrate. The foundation of all work in this paper rests on four systems that made monitoring progressively more programmable. NetFlow [6] established the paradigm of flow-level measurement with per-flow counters and periodic export, but its fixed-function design meant operators could not adapt measurement to changing conditions. OpenFlow [12] introduced the separation of control and data planes, allowing a remote controller to install and read measurement rules dynamically—a step-change in flexibility, though the switch itself remained a simple counter. OpenSketch [23] was the first system to argue that measurement deserves its own programmable pipeline, separate from forwarding. Its three-stage architecture (hash, filter, count) on NetFPGA hardware demonstrated that different measurement tasks could share the same hardware through software-controlled configuration—a direct precursor to MCP's resource allocation idea. P4 [4] generalized this to the entire switch, making the parser, match-action tables, and output logic all programmable. This made sketches, INT, and custom telemetry feasible at line rate, but also created the resource competition between forwarding and measurement that motivates MCP.

Polling optimization. OpenNetMon [20] adaptively adjusts polling intervals for edge switches, reducing overhead while maintaining per-flow QoS visibility (throughput, delay, loss).

Its limitation is that it monitors only at edge switches and optimizes only polling frequency—it does not consider switch resource usage or alternative measurement primitives. PayLess [5] provides a controller-side monitoring middleware with a REST API, adapting polling schedules based on traffic dynamics. It demonstrates the accuracy–overhead tradeoff but does not address multi-controller deployments or data-plane programmability. Xu *et al.* [21] reduce the number of statistics requests by aggregating flows into wildcard groups, cutting both control bandwidth and switch processing delay. Their work explicitly models request cost as an optimization variable but does not consider competing monitoring tasks or switch-side resource limits beyond processing delay.

Placement and selection. FlowCover [17] formulates the problem of choosing which switches to poll as a weighted set-cover problem, minimizing communication cost while ensuring all flows are observed. It provides the strongest formal treatment of the placement problem among the OpenFlow-era papers, but it does not account for switch-side resource limits, distributed controllers, or heterogeneous device capabilities. Tian *et al.* [19] define measurement value as information gain for traffic-matrix estimation: each measurement is worth installing only if it increases the rank of the linear system. This is the sharpest formulation of measurement value in the corpus, but it applies only to traffic-matrix estimation, not to security or QoS tasks. FlowSpy [9] moves monitoring task placement into the P4 data plane with load balancing, bridging the gap between OpenFlow-era polling and P4-era in-switch monitoring. It does not address multi-objective allocation or closed-loop evaluation. HeteroSketch [2] is the first system to address network-wide sketch placement across heterogeneous devices, using a profiling tool to quantify each device's capabilities and an optimization framework to assign tasks. It reduces resource overhead by 20–60% and handles dynamic changes. Its focus is on sketch placement, not on the broader question of which measurement *type* (sketch vs. polling vs. sampling vs. INT) to use at each location. Tahaei *et al.* [18] address the gap between single-controller measurement systems and the reality of distributed-controller SDN, proposing synchronization-aware flow measurement to control overhead. Their work is the closest to recognizing that controller architecture constrains monitoring design, but it does not address P4 programmability or multi-objective allocation.

Sketch infrastructure. UnivMon [11] introduced the idea of a universal sketch that delays the binding between data-plane computation and specific monitoring tasks, achieving both generality and accuracy. It demonstrated a P4 implementation and network-wide coordination, laying the theoretical and practical groundwork for all subsequent sketch-based monitoring. SketchLib [13] identified the practical bottlenecks that make sketches infeasible on real hardware and provided correct-by-construction optimizations. Its key contribution is showing that implementation details (how hash functions map to hardware units, how counter widths interact with ALU constraints) materially affect both feasibility and accuracy. Sketchovsky [14] optimized ensembles of sketches through cross-instance resource sharing, making it feasible to run many measurement tasks on one switch. It tested on a real Tofino switch with forwarding logic co-resident, demonstrating practical deployment. OctoSketch [24] solved the multi-core aggregation problem for software switches, and SketchPlan [16] raised the abstraction to intent-level planning with ensemble-wide optimization. These five systems collectively provide the measurement primitives that MCP would allocate. Our contribution is orthogonal: we address which primitives to run where, not how to implement them.

Query-driven telemetry. Sonata [10] is the closest prior work to MCP in spirit: it partitions monitoring queries across the switch and a stream processor, explicitly considering where computation should happen. MCP adopts Sonata's partitioning idea as one of its candidate-generation strategies but adds three things Sonata lacks: explicit multi-resource budgets (TCAM, registers, CPU), competition between multiple monitoring tasks, and closed-loop evaluation against downstream outcomes.

Security monitoring. Yoon *et al.* [22] use betweenness centrality to select sampling switches for feeding an IDS, with controlled sample rates to avoid overwhelming the IDS capacity. Their work is specific to IDS feeding and does not consider how security sampling competes with QoS or estimation tasks for switch resources. El Sayed *et al.* [7] show that SDN-native features and deep-learning classification outperform conventional approaches for DDoS detection, but their system operates entirely at the controller and does not consider P4-based data-plane monitoring or resource-constrained deployment.

Online optimization under resource constraints. Bandits with Knapsacks [3] provides the theoretical foundation for online learning under budget constraints, using primal–dual updates to balance exploration and exploitation while respecting resource limits. Foster *et al.* [8] extend contextual bandits to settings with multiple packing and covering constraints, providing the algorithmic template for MCP's multi-budget selector. CPO [1] offers constrained policy optimization for longer-horizon problems, but its weight and complexity make it better suited as a later-stage upgrade once the basic MCP pipeline is validated.

9 Limitations

This paper is an analysis and a proposal, not a working system. MCP has not been implemented. The bandit-based selector has not been evaluated on any concrete instance, and we do not know whether it converges fast enough for production use. The churn penalty and epoch duration Δ are

open design parameters that require empirical tuning on real traffic.

Our literature analysis covers eighteen papers selected to span the major architectural positions from 2008 to 2024. It is not exhaustive. Work in adjacent areas—eBPF-based host monitoring, SmartNIC offloading, network verification, and INT/PINT protocol design—may partially address some of the gaps we identify.

The untested assumptions in Section 3.3 are identified by careful reading of the papers, not by running falsification experiments. We argue they are untested, not necessarily that they are wrong. Some may hold up under experimental scrutiny; others may reveal real gaps.

10 Conclusion

Network monitoring has moved through three phases. In the first phase, routers had fixed counters and fixed sampling protocols. Operators could read what the hardware vendor provided, but they could not change what was measured. In the second phase, SDN and OpenFlow made the control plane programmable: a controller could install and remove measurement rules on demand, and systems like OpenSketch showed that measurement deserved its own programmable pipeline. In the third phase, P4 made the entire data plane programmable: sketches, in-band telemetry, and custom parsers could all run at line rate on switch hardware.

Each phase solved the previous bottleneck and created a new one. Fixed routers could not adapt to changing traffic. OpenFlow could adapt, but the switch could only count. P4 can do much more than count, but now the question is which of its many capabilities to use, where, and under what constraints.

The literature has produced strong individual results for each piece of this puzzle. Adaptive polling systems like OpenNetMon and PayLess showed how to reduce the cost of asking switches for data. Placement systems like FlowCover and HeteroSketch showed how to choose which switches to query. Sketch infrastructure systems like UnivMon, SketchLib, Sketchovsky, and SketchPlan showed how to run many measurement tasks on hardware efficiently. Query-driven systems like Sonata showed how to split work between the switch and a stream processor.

But these results do not compose. Each system optimizes one objective and treats the rest as externalities. A system that minimizes polling cost may overload the controller. A system that maximizes traffic-estimation quality may miss attack traffic. A system that loads an ensemble of sketches may exhaust pipeline stages needed for forwarding. A deployed network has QoS monitoring, traffic estimation, and attack detection all running at the same time, on the same switches, competing for the same TCAM, registers, and pipeline stages. Nobody has built the layer that manages this competition.

MCP is our proposal for that layer. It is a Measurement Control Plane that lives in the control plane, reads switch state through standard interfaces (P4Runtime, gNMI), and decides every second which measurement actions to deploy on which switches. It uses a constrained contextual bandit with shadow prices to balance accuracy, latency, and reconfiguration cost under hard resource budgets. It absorbs every major technique from the literature—adaptive polling, optimized placement, wildcard aggregation, centrality sampling, sketch ensembles, and query partitioning—as candidate actions in its allocation framework.

The optimization is solvable. The real difficulty is evaluation. Proving that MCP works requires closing the loop: not just showing that telemetry quality improves, but showing that the downstream application—the traffic engineer, the attack mitigator, the QoS controller—actually makes better decisions as a result. This closed-loop evaluation is the standard the field needs to adopt, and it is the open problem we leave for future work.

References

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proc. ICML*, 2017.

[2] Anup Agarwal, Zaoxing Liu, and Srinivasan Seshan. HeteroSketch: Coordinating network-wide monitoring in heterogeneous and dynamic networks. In *Proc. 19th USENIX NSDI*, 2022.

[3] Ashwinkumar Badanidiyuru, Robert Kleinberg, and Aleksandrs Slivkins. Bandits with knapsacks. *J. ACM*, 65(3):1–55, 2018.

[4] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. *ACM SIGCOMM Comput. Commun. Rev.*, 44(3):87–95, 2014.

[5] Shihabur Rahman Chowdhury, Md. Faizul Bari, Reaz Ahmed, and Raouf Boutaba. PayLess: A low cost network monitoring framework for software defined networks. In *Proc. IEEE/IFIP NOMS*, 2014.

[6] Benoit Claise. RFC 3954: Cisco systems NetFlow services export version 9. IETF RFC 3954, 2004.

[7] Mai S. El Sayed, Nhien-An Le-Khac, Marianne A. Azer, and Anca D. Jurcut. A flow-based anomaly detection approach with feature selection method against DDoS attacks in SDNs. *IEEE Trans. Cognitive Commun. Networking*, 2022.

[8] Dylan J. Foster, Noah Golowich, and Yanjun Han. Contextual bandits with packing and covering constraints: A modular Lagrangian approach via regression. In *Proc. COLT*, 2023.

[9] Xin Guan and Bin Shen. FlowSpy: An efficient network monitoring framework using P4 in software-defined networks. In *Proc. IEEE ICC*, 2019.

[10] Arpit Gupta, Rob Harrison, Marco Canini, Nick Feamster, Jennifer Rexford, and Walter Willinger. Sonata: Query-driven streaming network telemetry. In *Proc. ACM SIGCOMM*, pages 357–371, 2018.

[11] Zaoxing Liu, Antonis Manousis, Gregory Vorsanger, Vyas Sekar, and Vladimir Braverman. One sketch to rule them all: Rethinking network flow monitoring with UnivMon. In *Proc. ACM SIGCOMM*, pages 101–114, 2016.

[12] Nick McKeown, Tom Anderson, Hari Balakrishnan, Guru Parulkar, Larry Peterson, Jennifer Rexford, Scott Shenker, and Jonathan Turner. OpenFlow: Enabling innovation in campus networks. *ACM SIGCOMM Comput. Commun. Rev.*, 38(2):69–74, 2008.

[13] Hun Namkung, Zaoxing Liu, Daehyeok Kim, Vyas Sekar, and Peter Steenkiste. SketchLib: Enabling efficient sketch-based monitoring on programmable switches. In *Proc. 19th USENIX NSDI*, pages 743–759, 2022.

[14] Hun Namkung, Zaoxing Liu, Daehyeok Kim, Vyas Sekar, and Peter Steenkiste. Sketchovsky: Enabling ensembles of sketches on programmable switches. In *Proc. 20th USENIX NSDI*, pages 1273–1292, 2023.

[15] P4.org. $P4_{16}$ portable switch architecture (PSA), 2021.

[16] Milind Srivastava, Shao-Tse Hung, Hun Namkung, Kate Ching-Ju Lin, Zaoxing Liu, and Vyas Sekar. Raising the level of abstraction for sketch-based network telemetry with SketchPlan. In *Proc. ACM IMC*, 2024.

[17] Zhiyang Su, Ting Wang, Yu Xia, and Mounir Hamdi. FlowCover: Low-cost flow monitoring scheme in software defined networks. In *Proc. IEEE GLOBECOM*, 2014.

[18] Hamed Tahaei, Rosli Salleh, Mohd Faizal Ab. Razak, Kwangman Ko, and Nor Badrul Anuar. Cost effective network flow measurement for SDN: A distributed controller scenario. *IEEE Access*, 2018.

[19] Yuan Tian, Lingjun Chen, and Chin-Tau Lea. An SDN-based traffic matrix estimation framework. *IEEE Trans. Netw. Service Manag.*, 2018.

[20] Niels L.M. van Adrichem, Christian Doerr, and Fernando A. Kuipers. OpenNetMon: Network monitoring in OpenFlow software-defined networks. In *Proc. IEEE/IFIP NOMS*, 2014.

[21] Hongli Xu, Zhuolong Yu, Chen Qian, Xiang-Yang Li, and Zuohua Liu. Minimizing flow statistics collection cost of SDN using wildcard requests. *IEEE/ACM Trans. Networking*, 2017.

[22] Changhoon Yoon, Taejune Ha, Seungwon Kim, and Hyuk Lim. Scalable traffic sampling using centrality measure on software-defined networks. In *Proc. IEEE ICC*, 2017.

[23] Minlan Yu, Lavanya Jose, and Rui Miao. Software defined traffic measurement with OpenSketch. In *Proc. 10th USENIX NSDI*, pages 29–42, 2013.

[24] Yinda Zhang, Peiqing Chen, and Zaoxing Liu. OctoSketch: Enabling real-time, continuous network monitoring over multiple cores. In *Proc. 21st USENIX NSDI*, pages 1621–1639, 2024.